

PRACTICAL SESSION-7

Task 1 : Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

```
vennela_07@VennelaPC: ~/pr × + v
krishnasree08@krishnasree08: ~$ mkdir practical7
krishnasree08@krishnasree08: ~$ cd practical7
krishnasree08@krishnasree08: ~/practical7$ nano task1.c
krishnasree08@krishnasree08: ~/practical7$ ls
task1.c
krishnasree08@krishnasree08: ~/practical7$ gcc task1.c -o task1
krishnasree08@krishnasree08: ~/practical7$ ls
task1.c
krishnasree08@krishnasree08: ~/practical7$ ./task1
Linux Process Address Space
-----
Address of code      : 0x5db85e8ed1c9
Address of global    : 0x5db85e8f0010
Address of static    : 0x5db85e8f0014
Address of heap      : 0x5db8714212a0
Address of stack     : 0x7ffcb44c375c
```

```
GNU nano 7.2 task1.c
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;

void code_function() {
    printf("Address of code function : %p\n", (void *)code_function);
}

int main() {
    static int static_var = 20;
    int stack_var = 30;
    int *heap_var = (int *)malloc(sizeof(int));

    *heap_var = 40;

    printf("Linux Process Address Space\n");
    printf("-----\n");

    printf("Address of code      : %p\n", (void *)code_function);
    printf("Address of global    : %p\n", (void *)&global_var);
    printf("Address of static     : %p\n", (void *)&static_var);
    printf("Address of heap       : %p\n", (void *)heap_var);
    printf("Address of stack      : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

Task 2 : Make use of the /proc/maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
krishnasree08@krishnasree08: ~/$ nano task2.c
krishnasree08@krishnasree08: ~/$ gcc task2.c -o task2
krishnasree08@krishnasree08: ~/$ ./task2
Process ID: 44249
Press Enter to continue...
```

```
vennela_07@VennelaPC: ~/pr x + v
GNU nano 7.2 task2.c *
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process ID: %d\n", getpid());

    printf("Press Enter to continue...\n");
    getchar();

    printf("\n/proc/%d/maps\n", getpid());

    char command[100];
    sprintf(command, "cat /proc/%d/maps", getpid());

    system(command);

    return 0;
}
```